Lecture 6: Cross-validation, maybe nonparametrics

Max G'Sell
Machine Learning I: 46-926

November 16, 2020

Announcements

- There will be a survey soon about how the quizzes are working. Please let me know.
- As a reminder, quizzes come out Friday mornings and are due Sunday nights at 11pm.
- TA office hours are at 9am and 8pm Eastern on Tuesdays. Let me know if the schedule is not working.

Today: Cross-validation, starting nonparametrics

- A few comments on the quiz
- Model selection, cross-validation
- Less biased than linear regression: non-parametrics (start)

Review: Supervised learning

Observe $(X_1, Y_1), \ldots (X_n, Y_n) \overset{iid}{\sim} \mathcal{P}$. Estimate $\hat{\mu}(x)$ so that $\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X))$ is small for new $(X, Y) \sim \mathcal{P}$.

If we are minimizing expected squared prediction error:

- If we actually knew $\mathcal{P}$, the best we could do is

$$\mu(x) = \mathbb{E}(Y|X = x)$$

- If we are estimating $\hat{\mu}$, then

$$\mathbb{E}\left(\mathcal{L}(Y, \hat{\mu}(X)) \mid X = x\right) = \sigma_x^2 + \text{bias}(\hat{\mu}(x))^2 + \text{var}(\hat{\mu}(x))$$

Review: Linear regression

We can think of linear regression as one way to approximate $\mathbb{E}(Y|X = x)$ across x values, imagining:

$$\mathbb{E}(Y|X = x) \approx x^T \beta$$

This can work well if the linear model is not a bad approximation.

However, we noticed that, as the number of variables, p, grows large, the variance of linear regression grows rapidly. This leads to poor predictions even with the linear model is true!

The lasso

The lasso[1] estimate is defined as

$$\hat{\beta}^{\text{lasso}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \ \|y - X\beta\|_2^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

$$= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \ \underbrace{\|y - X\beta|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_1}_{\text{Penalty}}$$

The squared ℓ_2 penalty $\|\beta\|_2^2$ of ridge regression, has been replaced by an ℓ_1 penalty $\|\beta\|_1$. Even though these problems look similar, their solutions behave very differently

Note the name "lasso" is actually an acronym for: Least Absolute Selection and Shrinkage Operator

[1] Tibshirani (1996), "Regression Shrinkage and Selection via the Lasso"

$$\hat{\beta}^{\text{lasso}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \ \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

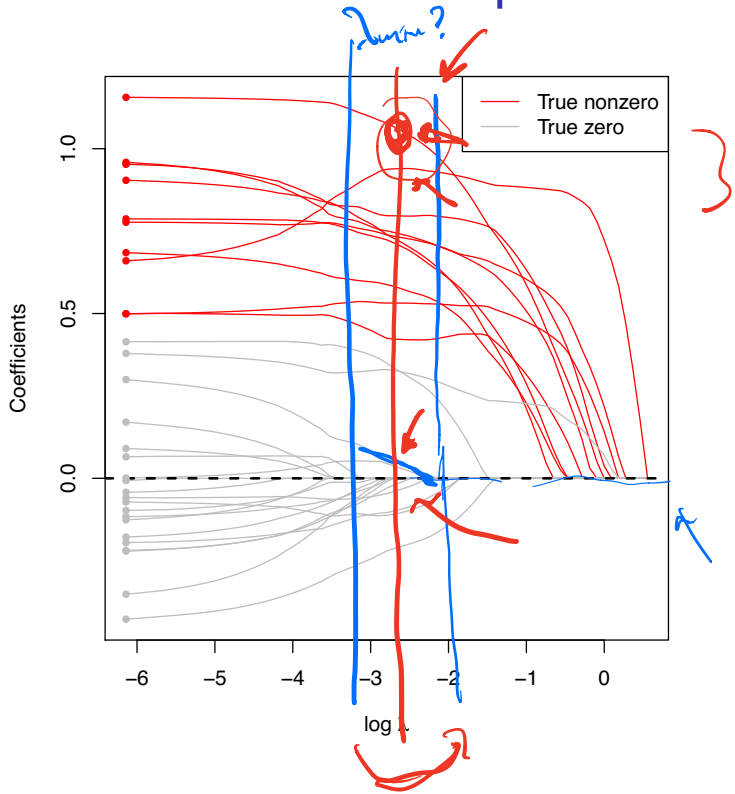
The tuning parameter λ controls the strength of the penalty.

- If $\lambda = 0$:
- As $\lambda \to \infty$:

For λ in between these two extremes, we are balancing two ideas: fitting a linear model of y on X, and shrinking the coefficients. But the nature of the ℓ_1 penalty causes some coefficients to be shrunken to zero exactly

This leads to variable selection!

Lasso coefficient paths



In the lasso, we get exact zeros! Moreover, the large coefficients are not impacted nearly as much!

It is often easier to view lasso results on the $\log \lambda$ scale.

Regularized regression

Least squares solves the problem:

$$\underset{\beta}{\operatorname{argmin}} \|y - X\beta\|_2^2$$

In regularized regression, we add a penalty to this squared error loss term

$$\underset{\beta}{\operatorname{argmin}} \|y - X\beta\|_2^2 + P_\lambda(\beta)$$

You've seen several examples: $P_\lambda(\beta) = \lambda \sum_j \beta_j^2$ (ridge regression) and $P_\lambda(\beta) = \lambda \sum_j |\beta_j|$ (lasso).

These penalties both "pull" β toward 0, causing it to be less variable. However, by making β smaller than the least squares estimate, they add bias.

The λ parameter determines how strong this penalty is. A larger λ leads to a higher bias and a smaller variance.

Regularized regression

$$\underset{\beta}{\operatorname{argmin}} \|y - X\beta\|_2^2 + P_\lambda(\beta)$$

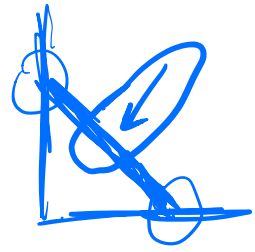
$P_\lambda(\beta) = \lambda \sum_j \beta_j^2$ (ridge regression) and $P_\lambda(\beta) = \lambda \sum_j |\beta_j|$ (lasso).

These penalties both "pull" β toward 0, causing it to be less

However, by making β smaller than the least squares estimate, they add

The λ parameter determines how strong this penalty is. A larger λ leads to a higher and a smaller

Extensions of the Lasso



Adding an L_1 penalty can induce sparsity in a variety of settings.

An entire industry has been developed around producing related methods and related theory.

We will see at least:

- Logistic regression with a lasso penalty
- The elastic net

$$\underset{\beta}{\operatorname{argmin}} \|Y - X\beta\|_2^2 + \lambda(1-\alpha)\|\beta\|_2^2 + \lambda\alpha\|\beta\|_1$$

Extensions of the Lasso

Adding an L_1 penalty can induce sparsity in a variety of settings.

An entire industry has been developed around producing related methods and related theory.

Other classic examples:

- The group lasso: setting groups of variables to zero
- Graphical lasso: estimates networks of dependence between variables
- Fused lasso: estimates piecewise-constant functions
- Trend filtering: estimates piecewise-polynomial functions
- Low-rank matrix decomposition: estimate a low-rank approximation to a matrix
- Sparse additive models
- Vector auto-regressive models

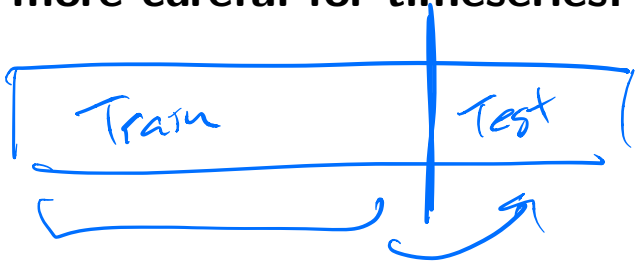
Model selection and generalization error

Regularized regression is one of the first settings you see where you must pick a particular model by setting a tuning parameter (λ here).

We know that λ trades off between bias and variance, and that we want to pick this tradeoff to minimize the *generalization error* for *new* samples.

How do we actually pick this λ?

[WARNING: We are going to talk about this for now in the i.i.d. setting. We need to be more careful for timeseries. We will come back to this.]

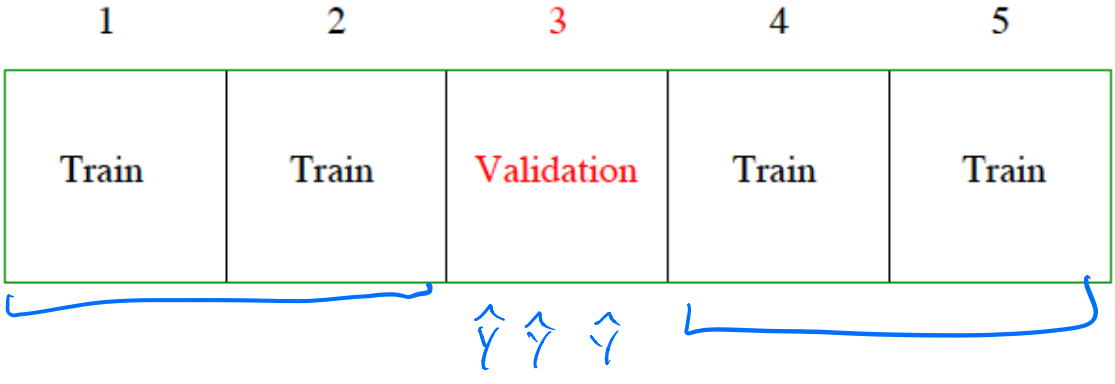


K-fold Cross-validation

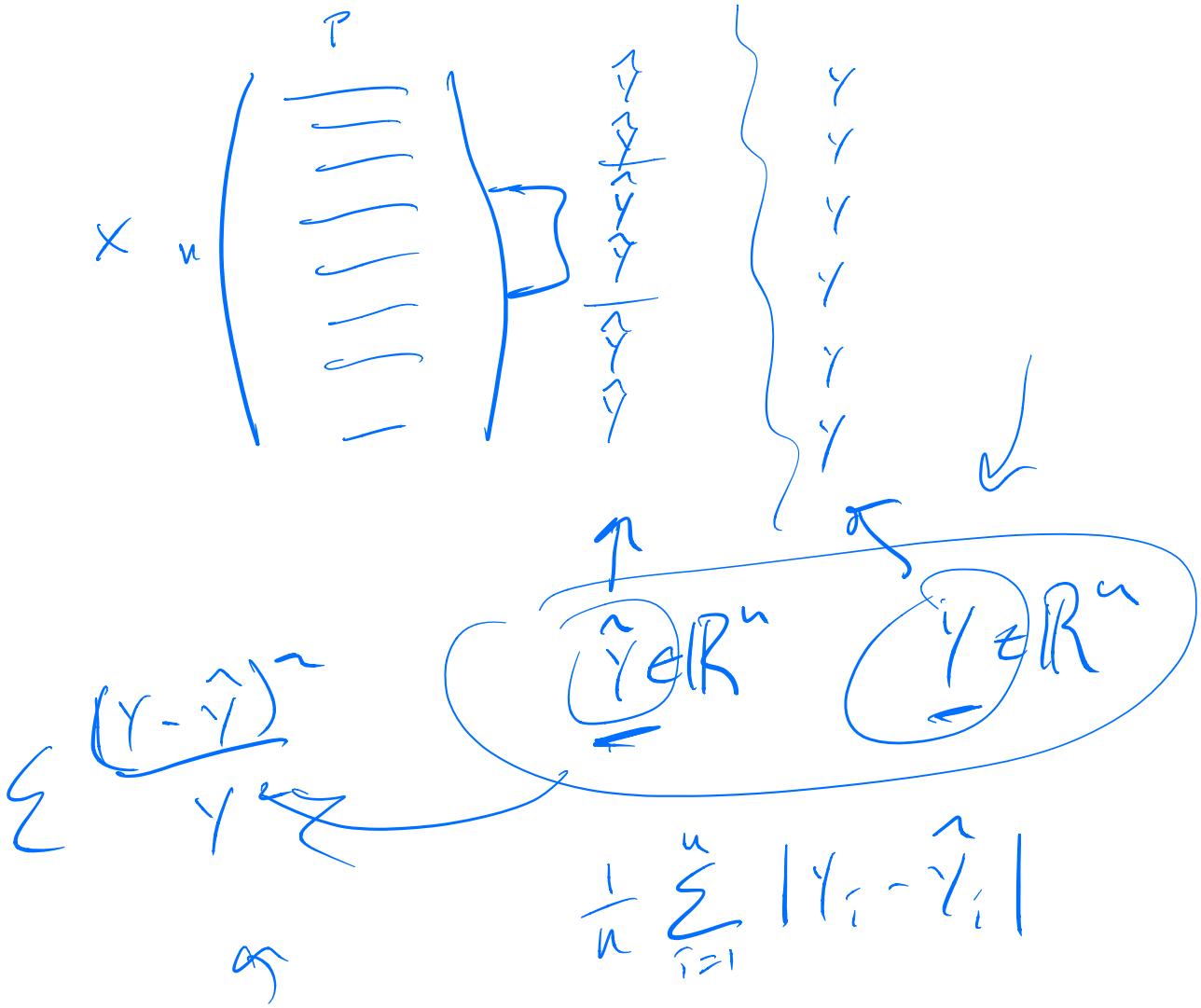
We want to estimate how well a method with each λ should do on *new* data points.

We don't have new data points, but we can hold out a few of our own.

We split the training pairs into K parts or "folds" (commonly $K = 5$ or $K = 10$)



K-fold cross validation considers training on all but the kth part, and then validating on the kth part, iterating over $k = 1, \ldots K$


$\hat{y} \in \mathbb{R}^n$
$y \in \mathbb{R}^n$
$(y - \hat{y})^2$
$\frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$

K-fold Cross-validation

We want to estimate how well a method with each λ should do on *new* data points.

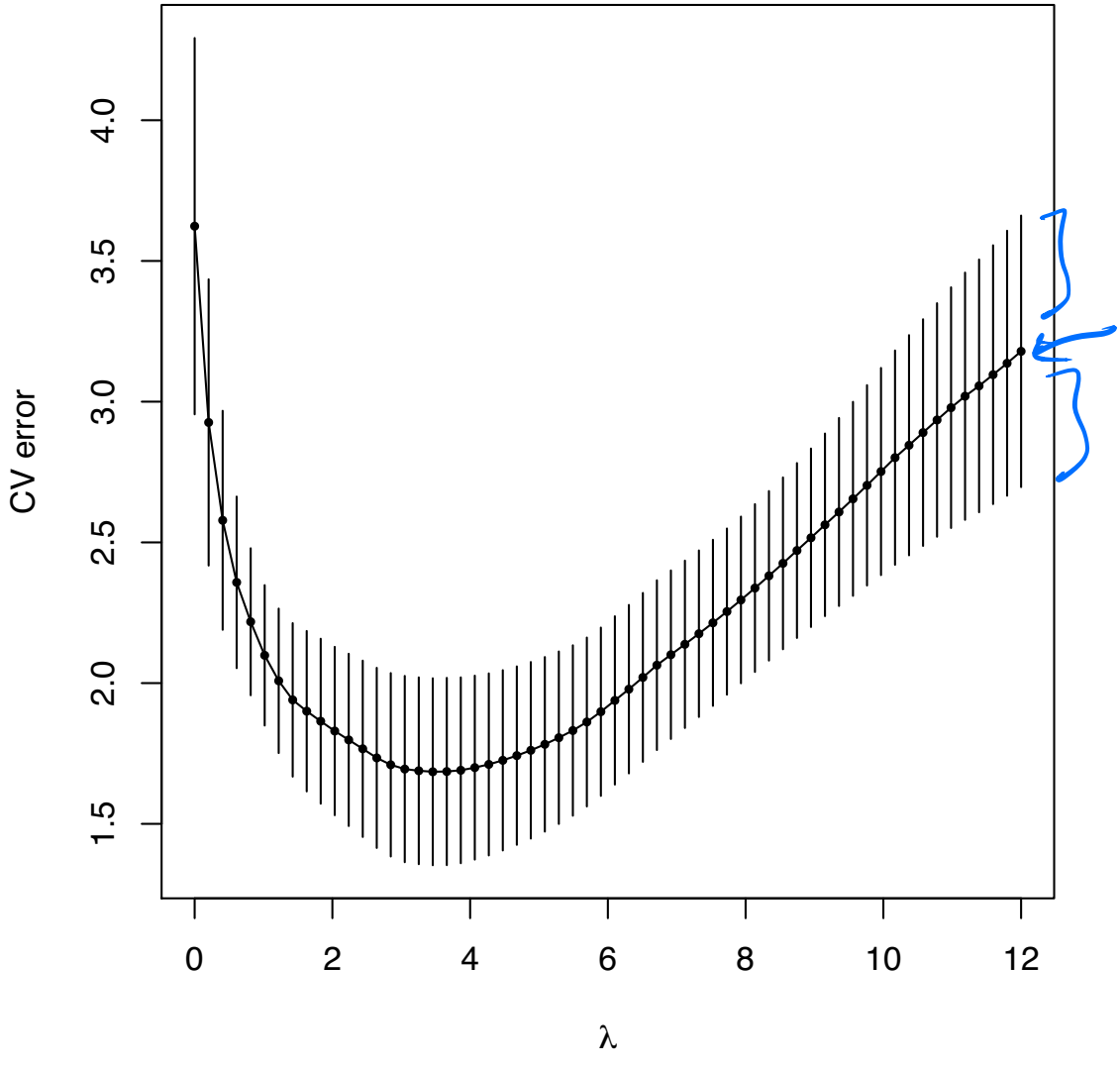
We don't have new data points, but we can hold out a few of our own.

K-fold Cross Validation:

1. Randomly split the data into K equal-sized parts ("folds")
2. Each part is used as the test set once, with the other $K-1$ parts combined into a training set
3. The error on all these test sets is averaged

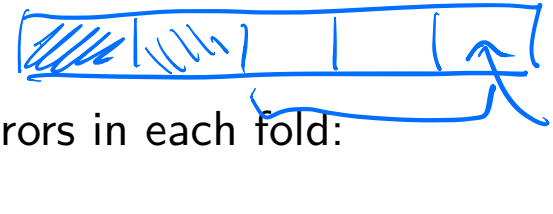
These errors are used to select a final model (λ value) to use. The model is refit on the entire data set for this λ value.

Usually $K = 5, 10,$ or n.


CV error
1.5
2.0
2.5
3.0
3.5
4.0
0
2
4
6
8
10
12
λ

Standard errors for cross-validation

One nice thing about K-fold cross-validation (for a small $K \ll n$, e.g., $K = 5$) is that we can estimate the standard deviation of $\mathrm{CV}(\theta)$, at each $\theta \in \{\theta_1, \ldots \theta_m\}$



First, we just average the validation errors in each fold:

$$\mathrm{CV}_k(\theta) = \frac{1}{n_k} e_k(\theta) = \frac{1}{n_k} \sum_{i \in F_k} \left(y_i - \hat{f}_\theta^{-k}(x_i)\right)^2$$

where n_k is the number of points in the kth fold

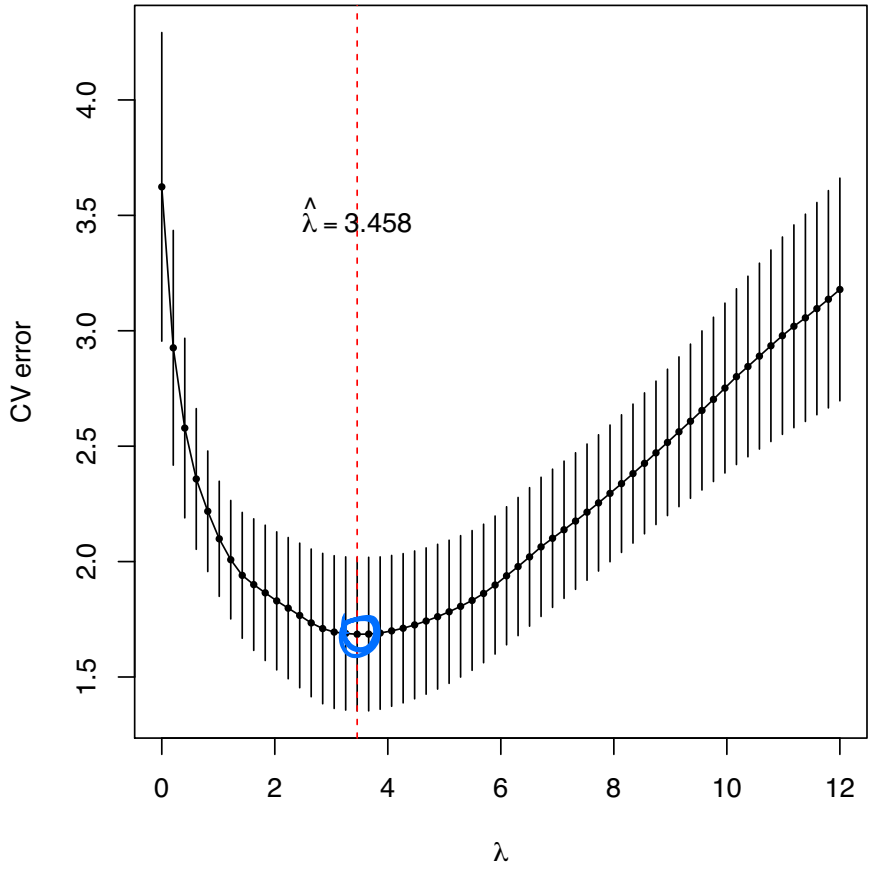
We then compute the sample standard deviation of $\mathrm{CV}_1(\theta), \ldots \mathrm{CV}_K(\theta)$,

$$\mathrm{SD}(\theta) = \sqrt{\mathrm{var}\big(\mathrm{CV}_1(\theta), \ldots \mathrm{CV}_K(\theta)\big)}$$

Finally we estimate the standard deviation of $\mathrm{CV}(\theta)$—called the standard error of $\mathrm{CV}(\theta)$—by $\mathrm{SE}(\theta) = \mathrm{SD}(\theta)/\sqrt{K}$

Example: choosing λ for the lasso (continued)

The cross-validation error curve from our lasso example, with $\pm$ standard errors:



The one standard error rule

The one standard error rule is an alternative rule for choosing the value of the tuning parameter, as opposed to minimizing the cross-validation error.

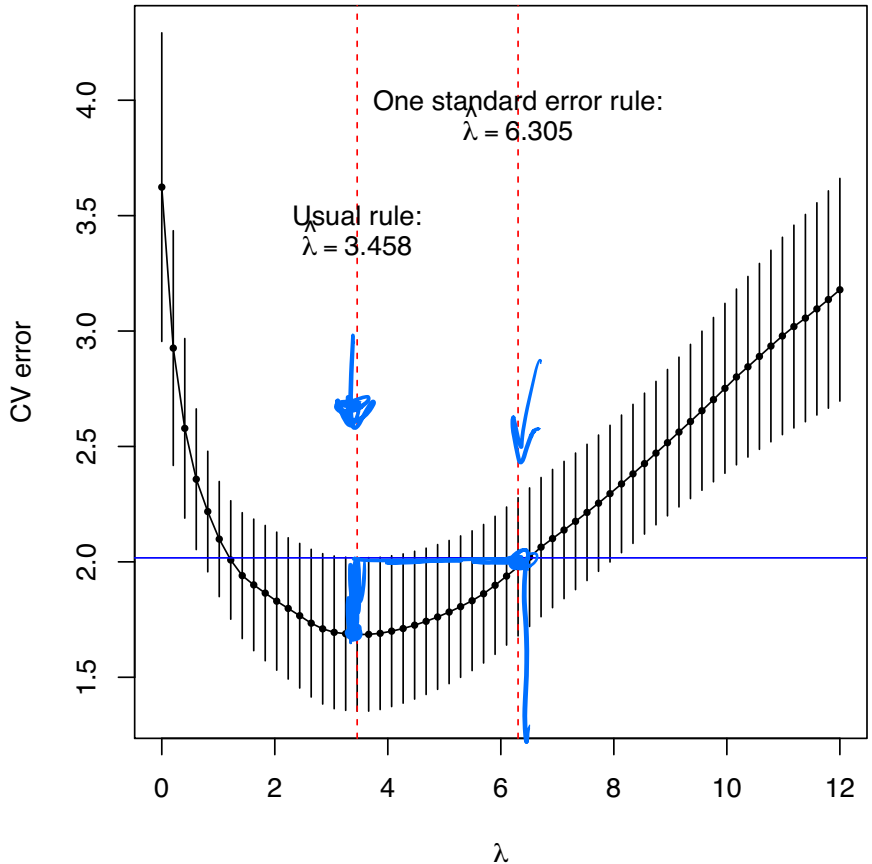
We find the usual minimizer $\hat{\theta}$, and then move θ in the direction of increasing regularization as much as we can, while keeping the CV error within one S.E. of $\mathrm{CV}(\hat{\theta})$:

$$\mathrm{CV}(\theta) \leq \mathrm{CV}(\hat{\theta}) + \mathrm{SE}(\hat{\theta})$$

Idea: "All else equal (up to one standard error), go for the simpler (more regularized) model"

Example: choosing λ for the lasso (continued)

Back to our lasso example: the one standard error rule chooses a larger value of λ (more regularization):

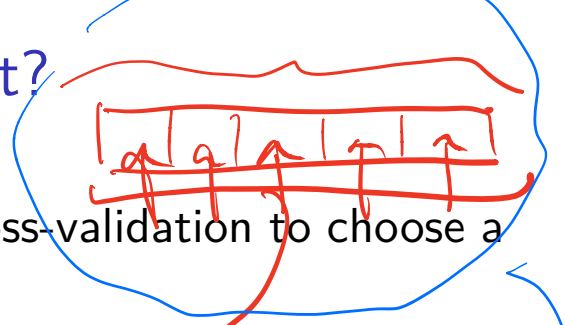


Prediction versus interpretation

Remember: cross-validation error estimates prediction error at any fixed value of the tuning parameter, and so by using it, we are implicitly assuming that achieving minimal prediction error is our goal.

However, this will tend to over-select variables, complicating our interpretation. The one standard error rule is a step in the direction of interpretability (and a hack).

What to do next?



What do we do next, after having used cross-validation to choose a value of the tuning parameter $\hat{\theta}$?

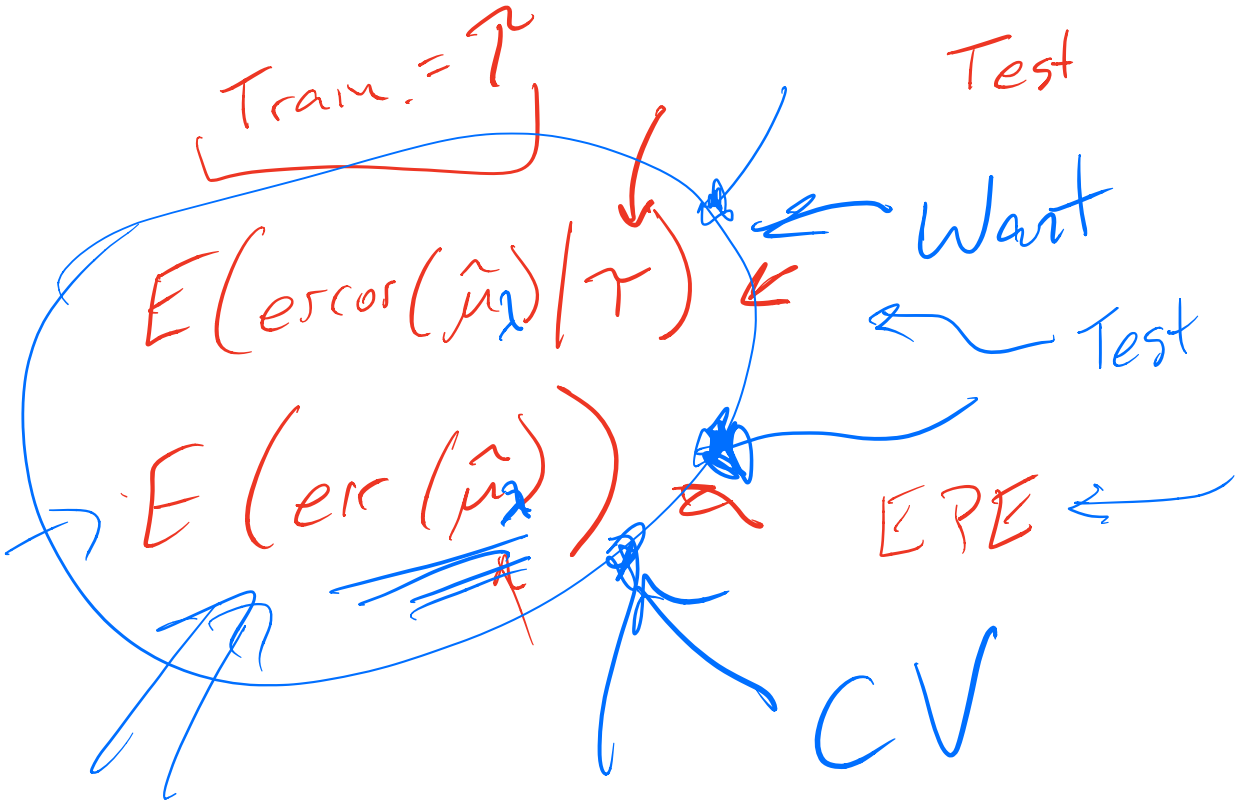
It may be an obvious point, but worth being clear: we now fit our estimator to the entire training set (x_i, y_i), $i = 1, \ldots n$, using the tuning parameter value $\hat{\theta}$

E.g., in the last lasso example, we resolve the lasso problem

$$\hat{\beta}^{\text{lasso}} = \underset{\beta \in \mathbb{R}^p}{\text{argmin}} \, \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

on all of the training data, with $\hat{\lambda} = 3.458$

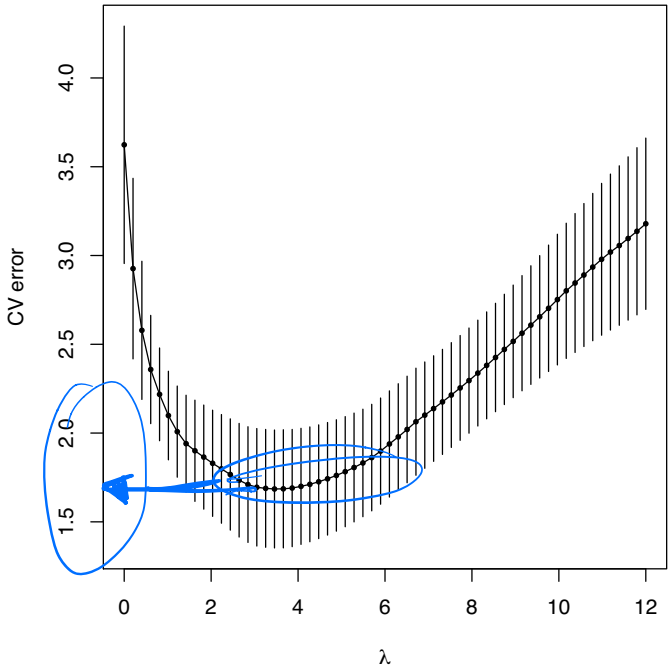
We can then use this estimator $\hat{\beta}^{\text{lasso}}$ to make future predictions


Train. = $\mathcal{T}$
Test
$E(\text{error}(\hat{\mu}_\lambda)|\mathcal{T})$
Wart
Test
$E(\text{err}(\hat{\mu}_\lambda))$
EPE
CV

Model assessment

Instead of just using the CV error curves to pick a value of the tuning parameter, suppose that we wanted to use cross-validation to actually estimate the prediction error values themselves

I.e., suppose that we actually cared about the values of the CV error curve, rather than just the location of its minimum



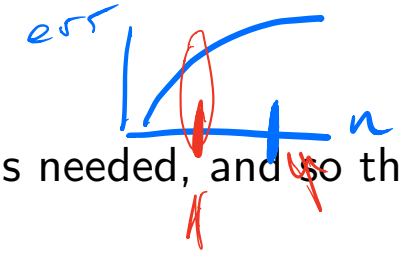
This is called model assessment (as opposed to model selection)

Model assessment

Why would we want to do this? Multiple reasons:

- Report estimate for prediction error to collaborator
- Compare estimated prediction error to known prediction errors of other estimators
- Answering the question: is my estimator $\hat{f}$ working at all?
- Choosing between different estimators (each possibly having their own tuning parameters)

Choice of K



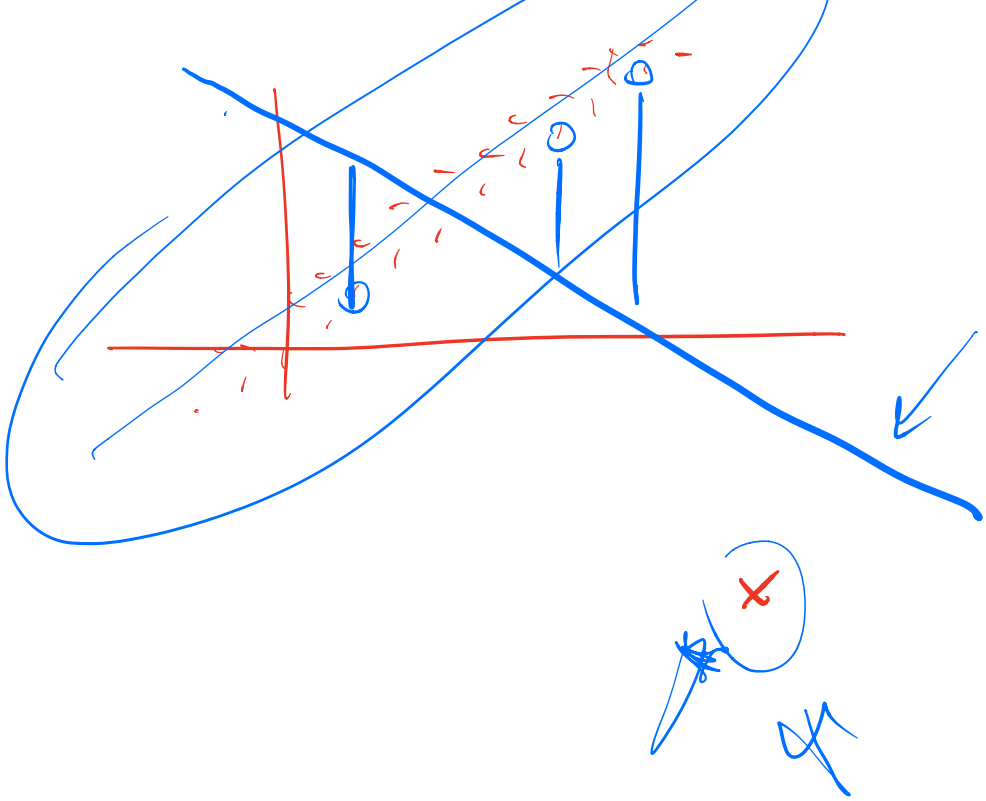
The larger the choice of K, the more iterations needed, and so the more computation.

Aside from computation, the choice of K affects the quality of our cross-validation error estimates for model assessment. Consider:

- $K = 2$: split-sample cross-validation. Our CV error estimates are going to be biased upwards, because we are only training on half the data each time
- $K = n$: leave-one-out cross-validation. Our CV estimates

$$\mathrm{CV}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - \hat{f}_\theta^{-i}(x_i) \right)^2$$

are going to be heavily positively correlated, which can lead to high variance.



Choosing $K = 5$ or $K = 10$ seems to generally be a good tradeoff

- In each iteration we train on a fraction of (about) $(K-1)/K$ the total training set, so this reduces the bias
- There is less overlap between the training sets across iterations, so the terms in

$$\mathrm{CV}(\theta) = \frac{1}{n} \sum_{k=1}^{K} \sum_{i \in F_k} \left(y_i - \hat{f}_\theta^{-k}(x_i)\right)^2$$

  are not as correlated, and the error estimate has a smaller variance

Leave-one-out shortcut

Recall that leave-one-out cross-validation is given by

$$\mathrm{CV}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - \hat{f}_\theta^{-i}(x_i) \right)^2$$

where $\hat{f}_\theta^{-i}$ is the estimator fit to all but the ith training pair (x_i, y_i)

Suppose that our tuning parameter is $\theta = \lambda$ and $\hat{f}_\lambda$ is the ridge regression estimator

$$\hat{f}_\lambda(x_i) = x_i^T \hat{\beta}^{\mathrm{ridge}} = x_i^T (X^T X + \lambda I)^{-1} X^T y$$

Then it turns out that

$$\frac{1}{n} \sum_{i=1}^{n} \left(y_i - \hat{f}_\lambda^{-i}(x_i) \right)^2 = \frac{1}{n} \sum_{i=1}^{n} \left[\frac{y_i - \hat{f}_\lambda(x_i)}{1 - S_{ii}} \right]^2$$

where $S = X(X^T X + \lambda I)^{-1} X^T$. This realization provides a huge computational savings!

A final warning about lasso

The lasso feels like magic, especially after you've tried to use regression in high-dimensions without it.

However, the lasso still selects a linear model. You will never do better than the best linear model using your variables could do for your problem.

The lasso cannot capture non-linearity or interactions (automatically). It cannot tell if your variables are poorly transformed.

Other methods can handle these problems. Do not neglect them.

What next?

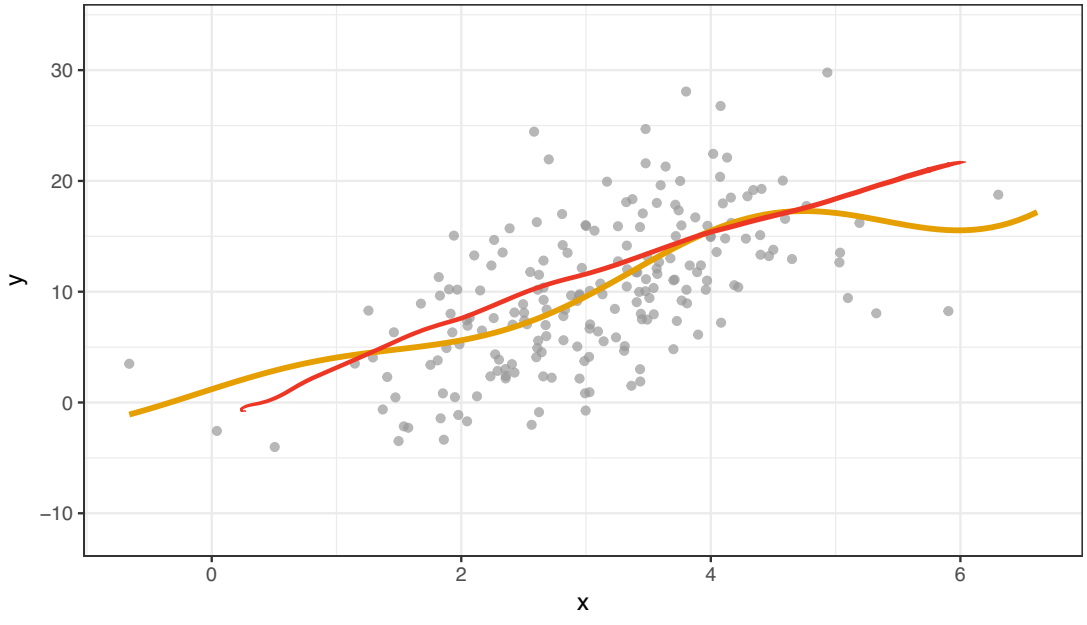
We started out by noting that we want to approximate $\mathbb{E}(Y|X = x)$.

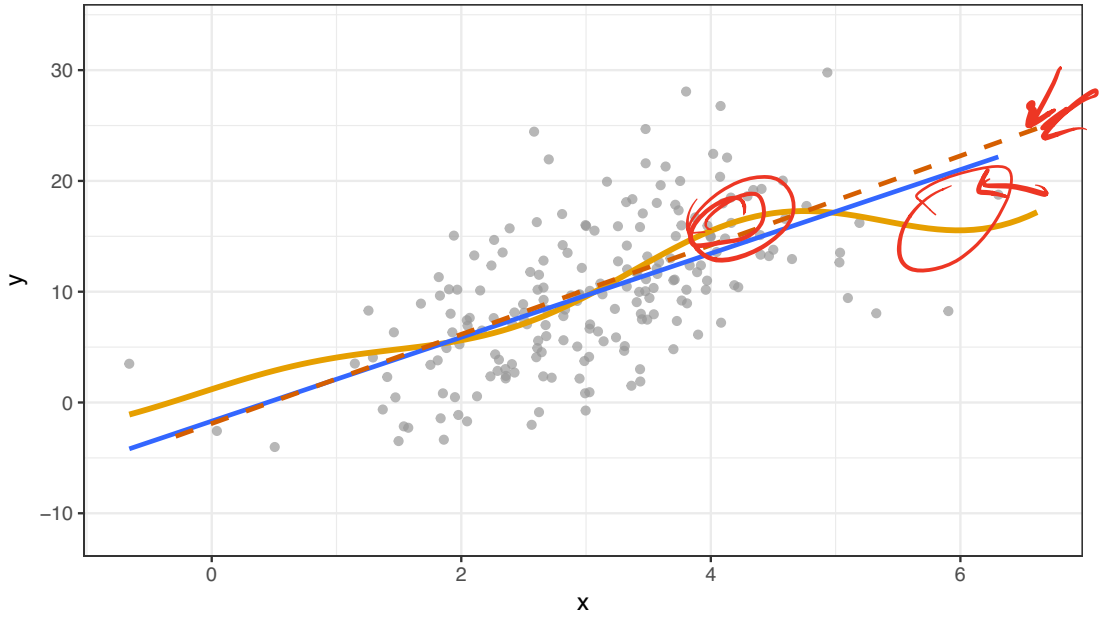
We thought about linear approximations, $\mathbb{E}(Y|X = x) = x^T\beta$, and showed that we can beat the least squares estimate with regularization to

- Reduce the variance (at the cost of bias).
- Improve interpretability by adding sparsity

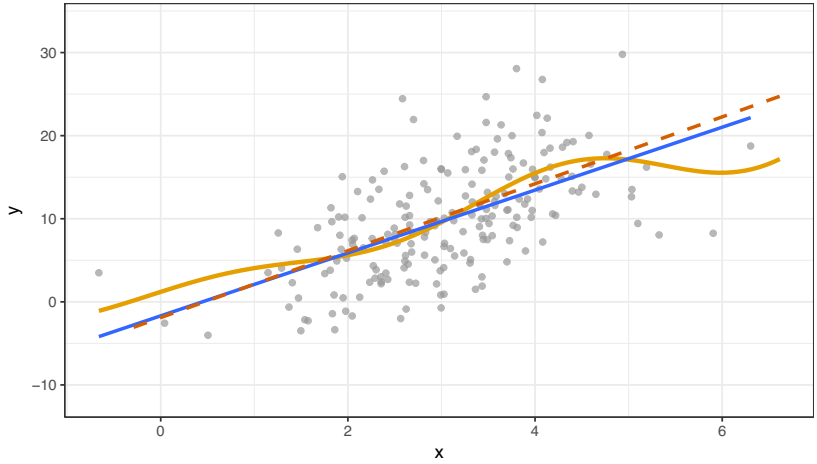
However, at the end of the day, we're still using something linear!

How do we move to something with lower bias instead of lower variance?


30
20
10
0
−10
y
0
2
4
6
x


30
20
10
0
−10
y
0
2
4
6
x

Consistency



One way to think about it: what happens as the number of samples grows?

If we use a linear model and take $n \to \infty$, what do we get?

Well then, what about a higher order polynomial to get some flexibility?

Consistency: An estimator $\hat{f}(x)$ is consistent if, as our sample size grows, $\hat{f}(x)$ converges to $f(x) = \mathbb{E}(Y|X = x)$.

Nonparametric regression

We will talk about two overall approaches to nonlinear regresion (for now). We begin with $X \in \mathbb{R}$, the one dimensional problem shown in the pictures above.

Kernel-based methods: We use a version of local averaging to approximate the function. Includes kernel smoothing, local linear regression. Closely related to k-nearest neighbors.

Spline-based methods. We fit a flexible piecewise polynomial to the data in a way that compromises between goodness-of-fit and smoothness.

k-nearest neighbors

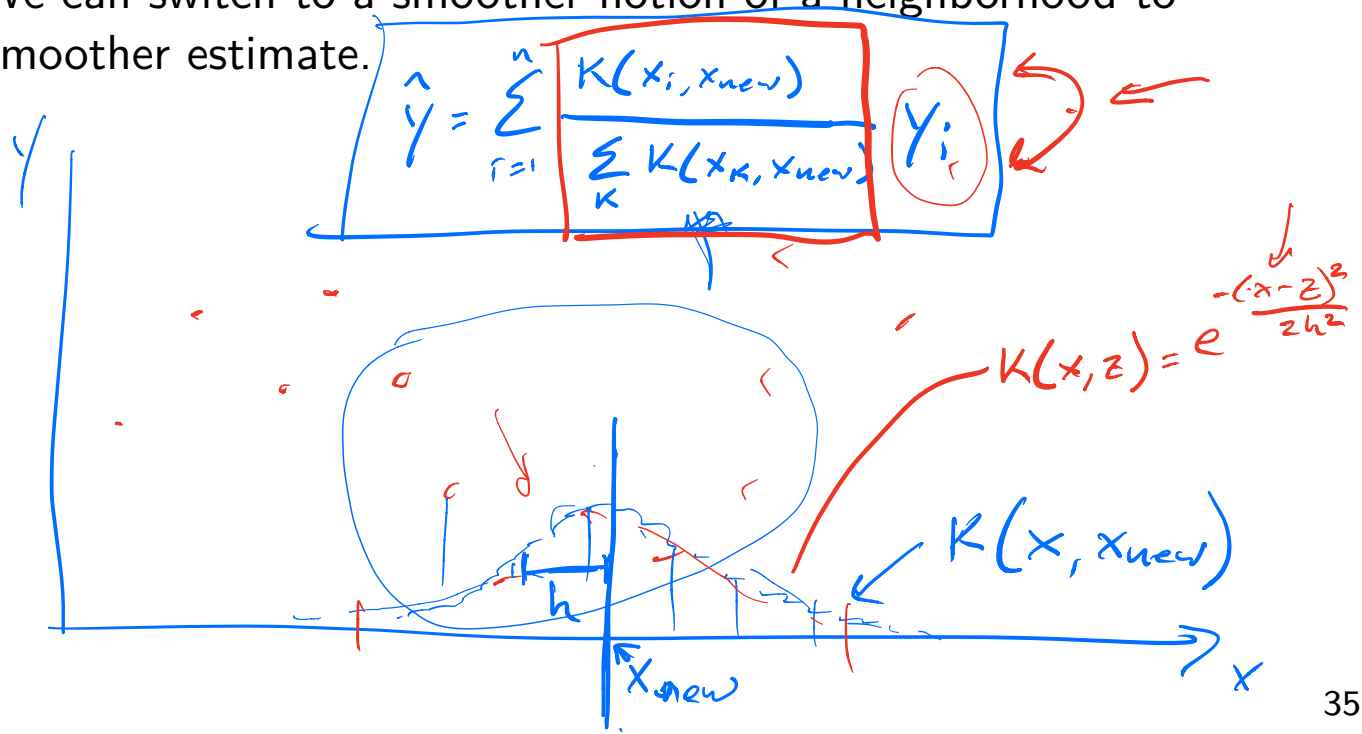
Recall that we want $\hat{\mu}(x)$ to be a good approximation of $\mathbb{E}(Y|X = x)$. What if we don't want to assume a strong form for the $\mathbb{E}(Y|X = x)$ function?

We might still be willing to assume that points near x are similar to x. We can use this to estimate the conditional mean!

Kernel regression

k-nearest neighbors can perform very well (in low dimensions or with nice distances). However, we might be bothered by how jagged the fit is!

This happens because being one of the k-nearest points is a jumpy idea. We can switch to a smoother notion of a neighborhood to get a smoother estimate.



Kernel regression

$$\hat{\mu}(x) = \sum_{i=1}^{n} y_i \frac{K(x_i, x)}{\sum_{j=1}^{n} K(x_j, x)}$$

$$K(x, x_j) = \frac{1}{\sqrt{2\pi h}} e^{-(x-x_j)^2/2h} \qquad \text{(Gaussian, example)}$$

Kernel regression gives a very simple way to get non-linear estimates of a function.

As we tune h, we determine how locally the fit looks at the training points, trading off bias and variance!

The method also has the advantage of being:

- Easy to prove things about
- Easy to generalize to other dimensions
- Easy to generalize to other methods

Kernel regression, example result

$$MSE(x) = \sigma^2(x) + h^4 \left(\frac{1}{2}\mu''(x) + \frac{\mu'(x)f'(x)}{f(x)} \right)^2 (\sigma_K^2)^2 + \frac{\sigma^2(x)R(K)}{nhf(x)} + o(h^4) + o((nh)^{-1})$$

Some simplification (and hiding):

$$MSE(x) = \sigma^2(x) + O(h^4) + O((nh)^{-1})$$

Kernel regression, higher dimensions

Suppose that we now have more than one dimension of X. How do we do nonparametric regression?

Just define a kernel to measure higher dimensional similarity!

$$K(x - x_j) = K_1(x^1 - x_j^1)K_2(x^2 - x_j^2) \cdots K_d(x^d - x_j^d)$$

Now everything works as before! Note that each kernel piece can have its own bandwidth...

However, we adapt terribly as dimension increases!

$$MSE - \sigma^2 =$$

Kernel regression: local linear regression

We can extend the idea of kernel regression, which just looks at local averages, to other local functions.

A classic example is local linear regression, which you have seen.

$$\min_{\beta_0,\beta} \frac{1}{n} \sum_{i=1}^{n} w_i(x)(y_i - \beta_0 - (x_i - x)\beta)^2$$

This is just weighted linear regression, so we already know the answer.

This leads to a smoother fit, can extrapolate better, and corrects a small term in the bias calculation.